PORTFOLIO · 作品集

徐亦优 Yiyou Xu

莫纳什大学 计算机科学 · Advanced Computer Science · 2027 届
yxuu0276@student.monash.edu · github.com/RGXeva

01
分布式电动车充电网络仿真
C · Open MPI · OpenMP · 集群
告警往返延迟 **0.81 → 0.32 ms**

02
并行素数搜索与性能优化
C · MPI · OpenMP · 集群
16 进程加速比 **11.4× → 15.6×**

03
Flippy Bit 网页游戏
TypeScript · RxJS · Vitest
87 个测试，**21/21** 个注入故障被捕获

04
MiniZinc-Lite 解析与求解器
Haskell · 解析器组合子
完整 BNF 文法，**30 + 22** 个测试模型

05
量化研究工具链
Rust · Python · LEAN
2025.12 起独立开发，有实盘经验

06
GPU 图像旋转
CUDA · Numba
2048^2 图像端到端快 **5.0×**

01 分布式电动车充电网络仿真

两人课程项目 · 主要负责人，完成约 90% 的工作 · 2026.09 – 2026.10 · 代码 github.com/RGXeva/ev-charging-sim

C　Open MPI　OpenMP　SLURM　Python

把一片充电站模拟成分布式传感网络：充电站拥堵时先问邻居，邻居也满了，再请基站推荐最近的空闲站。

我做了什么

- 比较 **8 种网络拓扑**后选定二维网格：每个充电站是一个 MPI 进程，用笛卡尔拓扑找上下左右邻居
- 每个充电桩是一个 OpenMP 线程，只写自己的槽位，**不加锁**也没有数据竞争
- 设计**非阻塞、按需触发**的邻居协议：用持久化接收随时应答邻居，避免死锁
- 基站在时间窗口内核对邻居状态，再按**曼哈顿环**由近及远找空闲站，一次返回同一距离的全部候选
- 实现优雅关停：基站广播终止，节点处理完手头事务后同步退出，回收线程、通信器和内存
- 负责架构设计、约 1500 行 C 代码、集群实验、PPT 和答辩展示



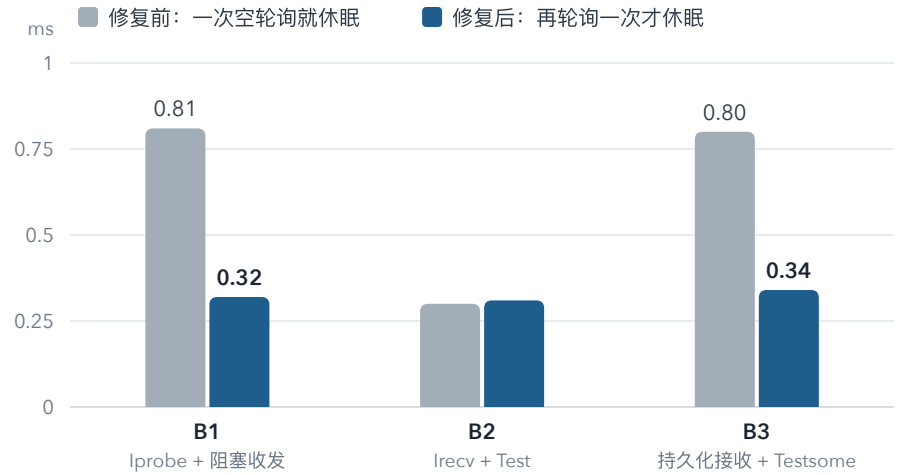


01 按实测选型，用数据找问题

分布式电动车充电网络仿真 · 实验与工程质量 CAAS 集群 ASan UBSan Python 分析脚本

发现轮询漏检，修复后告警延迟降低约六成

告警往返延迟中位数（ms），6×6 网格、37 个进程、3 台主机



一次空轮询就休眠，会漏掉轮询期间刚到的消息。改成"再轮询一次才休眠"后，B1 和 B3 的延迟都降了一半以上；B2 的接口本身会重新检查，所以本来就快。

三种节点方案，按实测选用 N2

节点方案	每节点每周期消息数	关停耗时
N1 · 每周期阻塞 Sendrecv	3.39	277 ms
N2 · 非阻塞、按需触发（选用）	**1.97**	**84 ms**
N3 · 每周期 Neighbor_allgather	3.39	279 ms

32/32
自动化测试在集群上全部通过

0
ASan、UBSan、LeakSanitizer 在自己代码中发现的错误

400+
次实验，本地与集群（最多 5 节点 × 16 核）

学校 CAAS 集群实机运行 · 3×3 网格 + 基站

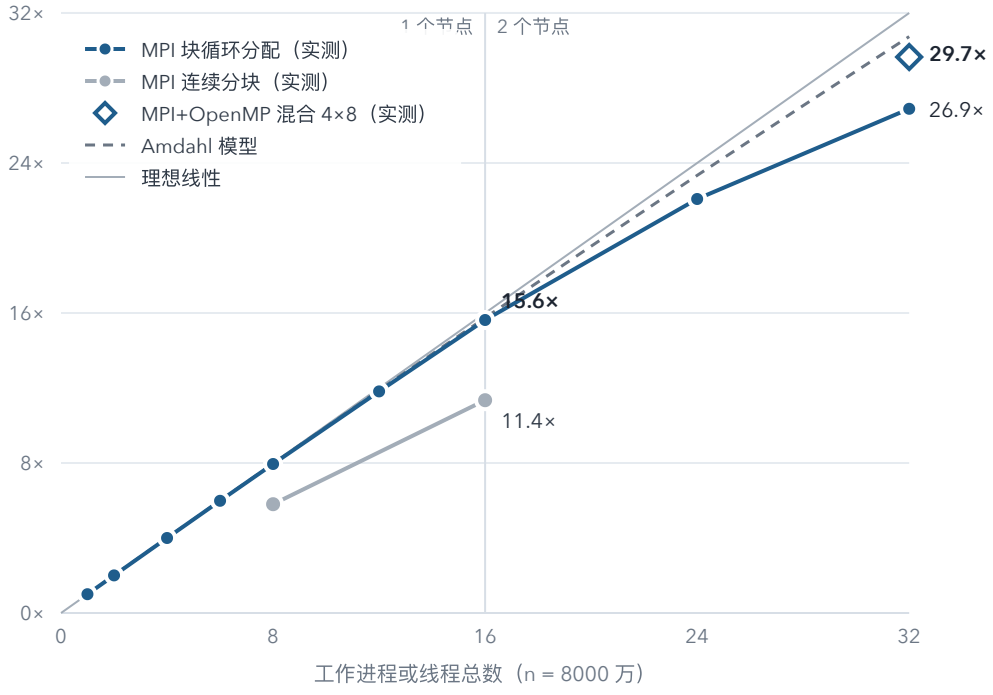
```
$ bash scripts/demo_caas.sh
srun: job 197921 has been allocated resources
[base t=  2.500s] alert #1 node 8 (2,2) free 1/6 -> suggest nearest available: 2(0,2) 4(1,1)
  6(2,0)
[base t=  6.500s] alert #2 node 6 (2,0) free 1/6 -> no available node nearby
[base t=  7.501s] alert #1 node 2 (0,2) free 0/6 -> suggest nearest available: 0(0,0)
  ...
[base] 3x3 N2/B2: 20.10 s, 23 alerts (20 candidates, 3 none nearby), 101 local redirects, 725
  messages
[base] neighbour exchange median 0.274 ms | alert round trip median 0.267 ms | shutdown
  96.497 ms
```

02 并行素数搜索：先找瓶颈，再修掉它

两人课程项目 · 主要负责人，完成约 90% 的工作 · 2026.08 – 2026.09　C　MPI　OpenMP　Pthreads　SLURM

加速比随工作进程数的变化

与串行版相比，集群实测中位数；虚线为用实测串行比例建立的 Amdahl 模型



从串行版本出发，依次写出 Pthreads、OpenMP、MPI 和 MPI+OpenMP 混合版本，每一步都先测量、再优化。

瓶颈一：负载不均

数越大，试除次数越多，连续分块让最后一个进程最累。改成 4096 个数一块、轮流分配后，最慢进程与平均进程之比 **1.41 → 1.00**，16 进程加速比 **11.4× → 15.6×**。

瓶颈二：0 号进程串行输出

试验中发现格式化输出全压在 0 号进程上。改成各进程格式化自己的结果、0 号只负责写文件后，试验加速比 **12.2× → 15.1×**。

873
次计时运行，覆盖 245 组配置

100%
输出与独立筛法结果逐一比对一致

0.16%
实测串行比例；单节点模型与实测相差不到 1%

03 Flippy Bit：函数式响应式网页游戏

个人课程作业 · 2026.08 – 2026.09 · 在线试玩 rgxeva.github.io/flippy-bit

TypeScript RxJS Vite Vitest



十六进制数字不断下落，玩家要在它落到虚线前，把底部 8 个二进制位翻成同一个值。

全部状态放在一条数据流里

五路事件
时钟、键盘、鼠标、暂停、重开

→

统一为动作
每个事件映射成 Action

→

merge + scan
折叠成唯一的游戏状态

→

render()
唯一有副作用的地方

游戏规则全部是纯函数，不碰 DOM、不读时钟，所以测试能在 Node 里直接驱动上百个时间步。

用变异测试检验测试本身

87
个测试覆盖纯函数模块和事件流

21/21
个故意注入的故障全部被测试抓到

+2
个测试：变异测试暴露出的盲区已补上

注入的故障包括判错目标、去掉速度上限、位序颠倒、原地修改状态等。其中"原地修改目标高度"第一轮没被抓到，原因是不可变性测试只检查了初始状态，于是补上了逐帧快照检查。

04 MiniZinc-Lite：约束模型解析与求解器

个人课程作业 · 2026.10 – 2026.11 Haskell 解析器组合子 BNF TypeScript 前端

读入一段约束建模语言，解析成语法树，检查名字和作用域，再求出全部可行解或最优解。

模型文本
.mzn 源文件
→
解析器组合子
按 BNF 逐条实现
→
语义检查
未声明、重复声明等文法表达不了的规则
→
语法树
声明、约束、目标
→
求解
全部可行解，或最大化、最小化

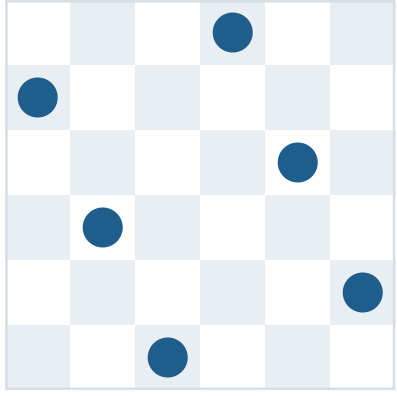
输入：6 皇后问题

```
int: n = 6;
array[1..n] of var 1..n: q;

constraint allDifferent(q);
constraint allDifferent(i in 1..n)(q[i] + i);
constraint allDifferent(i in 1..n)(q[i] - i);

solve satisfy;
```

求出的第一组解



```
[("q[1]",2),("q[2]",4),("q[3]",6),
 ("q[4]",1),("q[5]",3),("q[6]",5)]
… 共 4 组解
```

30
个样例模型，测试比对求解结果与标准答案

22
个非法模型，测试检查解析器能正确拒绝

配套一个网页前端：左侧输入模型，右侧实时显示求解结果，可一键保存解。

05 量化研究工具链

个人项目 · 独立负责设计、开发与研究 · 2025.12 至今　Rust　Python　QuantConnect LEAN　Docker　Tardis

从 2025 年 12 月起一个人持续开发两套研究工具链，并有实盘交易经验。下面只展示工程结构，策略细节和收益不公开。

数字资产研究工具链（Rust 重构）

行情归档
原始行情数据存档
→
标准化
统一不同来源的格式
→
历史回放
按事件时间重放市场
→
策略与因子
用 Tardis 数据挖因子、回测评估
→
执行与风控模拟
成交与风控的模拟检查

美股研究与回测平台（LEAN）

数据校验
行情数据质量检查
→
特征
研究用特征
→
信号
策略信号
→
交易成本
手续费等成本假设
→
订单处理接口
与券商实盘权限隔离

重点检查的工程问题

事件顺序 · 数量精度 · 手续费假设 · 异常处理 · 结果可复现。需求拆解、测试用例和代码审查与 AI 编程助手配合完成，生成的代码都经人工审查和测试验证。

因子研究

熟悉世坤 WorldQuant BRAIN 和 Qlib，做过因子构建和回测评估；用 Tardis 历史数据做因子挖掘。

06 GPU 图像旋转

两人课程项目 · 2026.10 CUDA Numba Python

用 GPU 并行旋转图片：一个线程负责一个输出像素，按逆映射去原图取像素，每个线程只写自己的位置，不需要原子操作。

- 16 × 16 的线程块，网格按图片尺寸向上取整；越界线程在边界检查处直接退出
- 计时覆盖"拷入显存 → 核函数 → 拷回内存"整条链路，和编译后的 CPU 版本对比
- 每次运行都与 CPU 结果核对，所有试验结果一致
- 图越大，数据搬运占比越低，GPU 优势越明显

端到端加速比（含显存拷贝）

旋转 30°，与单核 CPU 版本相比，5 次运行中位数

